METAGPT: META PROGRAMMING FOR A MULTI-AGENT COLLABORATIVE FRAMEWORK

Sirui Hong[1]*, **Mingchen Zhuge**[2]*, **Jiaqi Chen**[1], **Xiawu Zheng**[3], **Yuheng Cheng**[4], **Ceyao Zhang**[4], **Jinlin Wang**[1], **Zili Wang**, **Steven Ka Shing Yau**[5], **Zijuan Lin**[4], **Liyang Zhou**[6], **Chenyu Ran**[1], **Lingfeng Xiao**[1,7], **Chenglin Wu**[1]†, **Jürgen Schmidhuber**[2,8]

[1]DeepWisdom, [2]AI Initiative, King Abdullah University of Science and Technology,
[3]Xiamen University, [4]The Chinese University of Hong Kong, Shenzhen,
[5]Nanjing University, [6]University of Pennsylvania,
[7]University of California, Berkeley, [8]The Swiss AI Lab IDSIA/USI/SUPSI

ABSTRACT

Remarkable progress has been made on automated problem solving through societies of agents based on large language models (LLMs). Existing LLM-based multi-agent systems can already solve simple dialogue tasks. Solutions to more complex tasks, however, are complicated through logic inconsistencies due to cascading hallucinations caused by naively chaining LLMs. Here we introduce MetaGPT, an innovative meta-programming framework incorporating efficient human workflows into LLM-based multi-agent collaborations. MetaGPT encodes Standardized Operating Procedures (SOPs) into prompt sequences for more streamlined workflows, thus allowing agents with human-like domain expertise to verify intermediate results and reduce errors. MetaGPT utilizes an assembly line paradigm to assign diverse roles to various agents, efficiently breaking down complex tasks into subtasks involving many agents working together. On collaborative software engineering benchmarks, MetaGPT generates more coherent solutions than previous chat-based multi-agent systems. Our project can be found at https://github.com/geekan/MetaGPT.

1 INTRODUCTION

Autonomous agents utilizing Large Language Models (LLMs) offer promising opportunities to enhance and replicate human workflows. In real-world applications, however, existing systems (Park et al., 2023; Zhuge et al., 2023; Cai et al., 2023; Wang et al., 2023c; Li et al., 2023; Du et al., 2023; Liang et al., 2023; Hao et al., 2023; Zhou et al., 2023b) tend to oversimplify the complexities. They struggle to achieve effective, coherent, and accurate problem-solving processes, particularly when there is a need for meaningful collaborative interaction (Chen et al., 2024; Zhang et al., 2023a; Dong et al., 2023; Zhou et al., 2023a; Qian et al., 2023; Tang et al., 2023b; Hong et al., 2024).

Through extensive collaborative practice, humans have developed widely accepted Standardized Operating Procedures (SOPs) across various domains (Belbin, 2012; Manifesto, 2001; DeMarco & Lister, 2013). These SOPs play a critical role in supporting task decomposition and effective coordination. Furthermore, SOPs outline the responsibilities of each team member, while establishing standards for intermediate outputs. Well-defined SOPs improve the consistent and accurate execution of tasks that align with defined roles and quality standards (Belbin, 2012; Manifesto, 2001; DeMarco & Lister, 2013; Wooldridge & Jennings, 1998). For instance, in a software company, Product Managers analyze competition and user needs to create Product Requirements Documents (PRDs) using a standardized structure, to guide the developmental process.

Inspired by such ideas, we design a promising GPT-based Meta-Programming framework called MetaGPT that significantly benefits from SOPs. Unlike other works (Li et al., 2023; Qian et al., 2023), MetaGPT requires agents to generate structured outputs, such as high-quality requirements

*These authors contributed equally to this work.
†Chenglin Wu (alexanderwu@fuzhi.ai) is the corresponding author, affiliated with DeepWisdom.

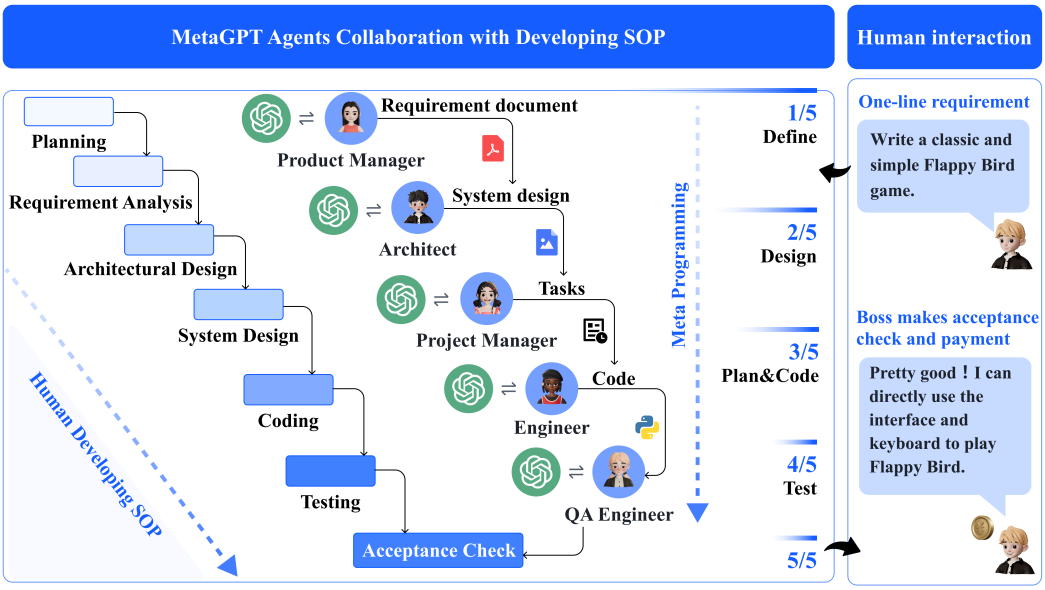

Figure 1: **The software development SOPs between MetaGPT and real-world human teams.** In software engineering, SOPs promote collaboration among various roles. MetaGPT showcases its ability to decompose complex tasks into specific actionable procedures assigned to various roles (e.g., Product Manager, Architect, Engineer, etc.).

documents, design artifacts, flowcharts, and interface specifications. The use of intermediate structured outputs significantly increases the success rate of target code generation. Because it helps maintain consistency in communication, minimizing ambiguities and errors during collaboration. More graphically, in a company simulated by MetaGPT, all employees follow a strict and streamlined workflow, and all their handovers must comply with certain established standards. This reduces the risk of hallucinations caused by idle chatter between LLMs, particularly in role-playing frameworks, like: "*Hi, hello and how are you?*" – Alice (Product Manager); "*Great! Have you had lunch?*" – Bob (Architect).

Benefiting from SOPs, MetaGPT offers a promising approach to meta-programming. In this context, we adopt meta-programming[1] as "programming to program", in contrast to the broader fields of meta learning and "learning to learn" (Schmidhuber, 1987; 1993a; Hochreiter et al., 2001; Schmidhuber, 2006; Finn et al., 2017).

This notion of meta-programming also encompasses earlier efforts like CodeBERT (Feng et al., 2020) and recent projects such as CodeLlama (Rozière et al., 2023) and WizardCoder (Luo et al., 2023). However, MetaGPT stands out as a unique solution that allows for efficient meta-programming through a well-organized group of specialized agents. Each agent has a specific role and expertise, following some established standards. This allows for automatic requirement analysis, system design, code generation, modification, execution, and debugging during runtime, highlighting how agent-based techniques can enhance meta-programming.

To validate the design of MetaGPT, we use publicly available HumanEval (Chen et al., 2021a) and MBPP (Austin et al., 2021) for evaluations. Notably, in code generation benchmarks, MetaGPT achieves a new state-of-the-art (SoTA) with 85.9% and 87.7% in Pass@1. When compared to other popular frameworks for creating complex software projects, such as AutoGPT (Torantulino et al., 2023), LangChain (Chase, 2022), AgentVerse (Chen et al., 2023), and ChatDev (Qian et al., 2023). MetaGPT also stands out in handling higher levels of software complexity and offering extensive functionality. Remarkably, in our experimental evaluations, MetaGPT achieves a 100% task completion rate, demonstrating the robustness and efficiency (time and token costs) of our design.

We summarize our contributions as follows:

[1] https://en.wikipedia.org/w/index.php?title=Metaprogramming

• We introduce MetaGPT, a meta-programming framework for multi-agent collaboration based on LLMs. It is highly convenient and flexible, with well-defined functions like role definition and message sharing, making it a useful platform for developing LLM-based multi-agent systems.

• Our innovative integration of human-like SOPs throughout MetaGPT's design significantly enhances its robustness, reducing unproductive collaboration among LLM-based agents. Furthermore, we introduce a novel executive feedback mechanism that debugs and executes code during runtime, significantly elevating code generation quality (e.g., 5.4% absolute improvement on MBPP).

• We achieve state-of-the-art performance on HumanEval (Chen et al., 2021a) and MBPP (Austin et al., 2021). Extensive results convincingly validate MetaGPT, suggesting that it is a promising meta-programming framework for developing LLM-based multi-agent systems.

2 Related Work

Automatic Programming The roots of automatic programming reach back deep into the previous century. In 1969, Waldinger & Lee (1969) introduced "PROW," a system designed to accept program specifications written in predicate calculus, generate algorithms, and create LISP implementations (McCarthy, 1978). Balzer (1985) and Soloway (1986) made efforts to advance automatic programming and identified potential methods to achieve it. Recent approaches use natural language processing (NLP) techniques (Ni et al., 2023; Skreta et al., 2023; Feng et al., 2020; Li et al., 2022; Chen et al., 2018; 2021b; Zhang et al., 2023a; Liu et al., 2023b; Tang et al., 2023a; Muennighoff et al., 2023). Automatic programming has grown into an industry delivering paid functions such as Microsoft Copilot. Lately, LLMs-based agents (Yao et al., 2022; Shinn et al., 2023; Lin et al., 2023) have advanced automatic programming development. Among them, ReAct (Yao et al., 2022) and Reflexion (Shinn et al., 2023) utilize a chain of thought prompts (Wei et al., 2022) to generate reasoning trajectories and action plans with LLMs. Both works demonstrate the effectiveness of the ReAct style loop of reasoning as a design paradigm for empowering automatic programming. Additionally, ToolFormer (Schick et al., 2023) and ToolLLM (Qin et al., 2023) can learn how to use external tools through simple APIs. The research most closely aligned with our work by Li et al. (2023) proposes a straightforward role-play framework for programming that involves communication between agents playing different roles. Qian et al. (2023) utilizes multiple agents for software development. Although existing papers (Li et al., 2023; Qian et al., 2023) have improved productivity, they have not fully tapped into effective workflows with structured output formats. This makes it harder to deal with complex software engineering issues.

LLM-Based Multi-Agent Frameworks Recently, LLM-based autonomous agents have gained tremendous interest in both industry and academia (Wang et al., 2023b; Zhou et al., 2023b; Zhang et al., 2023b). Many works (Chen et al., 2024; Wang et al., 2023c; Du et al., 2023; Zhuge et al., 2023; Hao et al., 2023; Akata et al., 2023; Tang et al., 2023b) have improved the problem-solving abilities of LLMs by integrating discussions among multiple agents. Stable-Alignment (Liu et al., 2023a) creates instruction datasets by deriving consensus on value judgments through interactions across a sandbox with LLM agents. Other works focus on sociological phenomena. For example, Generative Agents (Park et al., 2023) creates a "town" of 25 agents to study language interaction, social understanding, and collective memory. In the Natural Language-Based Society of Mind (NLSOM) (Zhuge et al., 2023), agents with different functions interact to solve complex tasks through multiple rounds of "mindstorms." Cai et al. (2023) propose a model for cost reduction by combining large models as tool makers and small models as tool users.

Some works emphasize cooperation and competition related to planning and strategy (Bakhtin et al., 2022); others propose LLM-based economies (Zhuge et al., 2023). These works focus on open-world human behavior simulation, while MetaGPT aims to introduce human practice into multi-agents frameworks. Besides, LLM-based agents face the challenges of "assistant repeated instruction" or "infinite loop of message" (Talebirad & Nadiri, 2023; Li et al., 2023). These challenges become more urgent in task-oriented collaborations, which require consistent and mutually beneficial interactions (Elazar et al., 2021; Wang et al., 2022; Jiang et al., 2023). This motivates our focus on applying advanced concepts such as Standard Operating Procedures in software development to multi-agent frameworks.

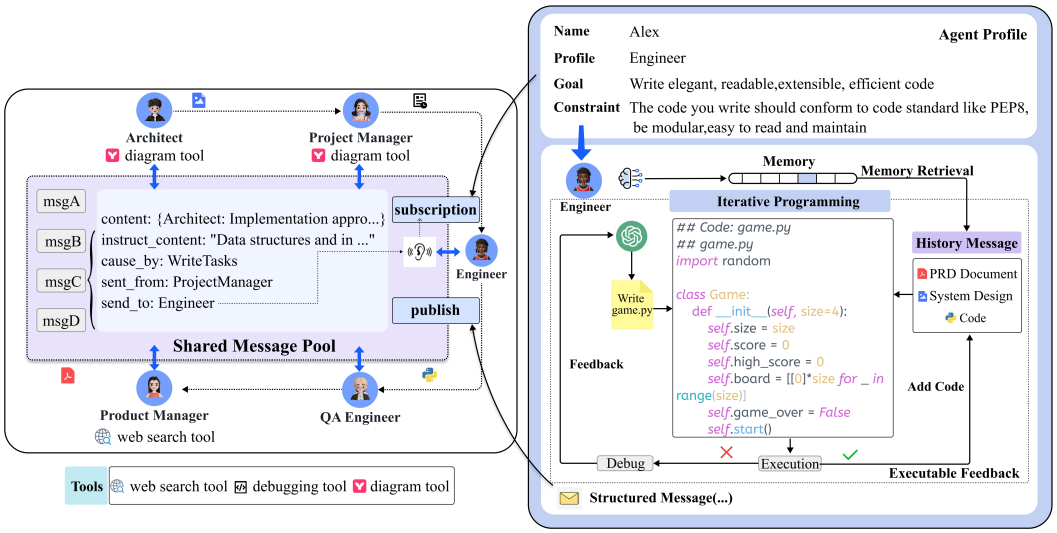

Figure 2: An example of the communication protocol (left) and iterative programming with executable feedback (right). **Left**: Agents use a shared message pool to publish structured messages. They can also subscribe to relevant messages based on their profiles. **Right**: After generating the initial code, the Engineer agent runs and checks for errors. If errors occur, the agent checks past messages stored in memory and compares them with the PRD, system design, and code files.

3 METAGPT: A META-PROGRAMMING FRAMEWORK

MetaGPT is a meta-programming framework for LLM-based multi-agent systems. Sec. 3.1 provides an explanation of role specialization, workflow and structured communication in this framework, and illustrates how to organize a multi-agent system within the context of SOPs. Sec. 3.2 presents a communication protocol that enhances role communication efficiency. We also implement structured communication interfaces and an effective publish-subscribe mechanism. These methods enable agents to obtain directional information from other roles and public information from the environment. Finally, we introduce executable feedback—a self-correction mechanism for further enhancing code generation quality during run-time in Sec. 3.3.

3.1 AGENTS IN STANDARD OPERATING PROCEDURES

Specialization of Roles Unambiguous role specialization enables the breakdown of complex work into smaller and more specific tasks. Solving complex tasks or problems often requires the collaboration of agents with diverse skills and expertise, each contributing specialized outputs tailored to specific issues.

In a software company, a Product Manager typically conducts business-oriented analysis and derives insights, while a software engineer is responsible for programming. We define five roles in our software company: Product Manager, Architect, Project Manager, Engineer, and QA Engineer, as shown in Figure 1. In MetaGPT, we specify the agent's profile, which includes their name, profile, goal, and constraints for each role. We also initialize the specific context and skills for each role. For instance, a Product Manager can use web search tools, while an Engineer can execute code, as shown in Figure 2. All agents adhere to the React-style behavior as described in Yao et al. (2022).

Every agent monitors the environment (*i.e.*, the message pool in MetaGPT) to spot important observations (*e.g.*,, messages from other agents). These messages can either directly trigger actions or assist in finishing the job.

Workflow across Agents By defining the agents' roles and operational skills, we can establish basic workflows. In our work, we follow SOP in software development, which enables all agents to work in a sequential manner.

Figure 3: A diagram showing the software development process in MetaGPT, emphasizing its significant dependence on SOPs. The more detailed demonstration can be found in Appendix B.

Specifically, as shown in Figure 1, upon obtaining user requirements, the Product Manager undertakes a thorough analysis, formulating a detailed PRD that includes User Stories and Requirement Pool. This serves as a preliminary functional breakdown. The structured PRD is then passed to the Architect, who translates the requirements into system design components, such as File Lists, Data Structures, and Interface Definitions. Once captured in the system design, the information is directed towards the Project Manager for task distribution. Engineers proceed to execute the designated classes and functions as outlined (detailed in Figure 2). In the following stage, the QA Engineer formulates test cases to enforce stringent code quality. In the final step, MetaGPT produces a meticulously crafted software solution. We provide a detailed schematic (Figure 3) and a concrete instance (Appendix B) of the SOP workflow in MetaGPT.

3.2 Communication Protocol

Structured Communication Interfaces Most current LLM-based multi-agent frameworks (Li et al., 2023; Zhuge et al., 2023; Zhang et al., 2023a; Park et al., 2023) utilize unconstrained natural language as a communication interface.

However, despite the versatility of natural language, a question arises: does pure natural language communication suffice for solving complex tasks? For example, in the telephone game (or Chinese